SOFTWARE ARCHITECTURE DOCUMENT

Turn Multi-Axis Kinematics

How the Turn Klipper fork lets the kinematics class define a printer's axes instead of hard-coding them, and how each machine is configured against it.

Status	Draft
Owner	Turn-Motion firmware
Last updated	17 September 2026
Applies to	`cartesianxy3z`, `cartesianxy3zr`, and the optional derived bed joint carriages
Baseline	Klipper3d/klipper @ `2d7717e3`
Related documents	`docs/Turn_Kinematics.md` - this document as text `FORK_CHANGES.md` - the file-by-file maintenance ledger, and the procedure for moving to a new upstream release

Purpose and scope

This document describes the motion architecture of the Turn Klipper fork: how many kinematic axes a printer has, which component decides that, and how the host software is written so the answer can differ between machines. It is the reference for changes to motion, homing, probing and g-code handling, and for configuring a machine against them.

Why the fork is based on current upstream

The previous fork was based on Klipper from April 2025. Upstream moved 705 commits beyond it, rewriting the lookahead queue and moving step generation into a new `motion_queuing` module. The fork's `toolhead.py` was by then an old upstream file carrying five-axis edits; it could not be rebased mechanically and was re-ported by hand onto current upstream.

Rebasing onto upstream is therefore an architectural constraint rather than housekeeping:

- **Divergence compounds.** Upstream restructures the motion path every few releases. A fork that carries edits inside those files pays for each restructuring it skipped, and eventually cannot rebase at all.
- **Upstream's tests apply to this code.** Because the stock kinematics still run through this motion core, upstream's own regression configurations exercise it rather than bypassing it.
- **Fixes and hardware support arrive without work**, provided the fork is a small stack on top rather than a snapshot beside.
- **Changes can be proposed upstream.** That is the only route by which the fork's footprint shrinks instead of growing.

Why the axis count is generalised rather than raised

Three machines are in scope: a gantry over a three-motor bed, the same bed with derived horizontal joint carriages, and the same machine with a rotary axis. Raising the hard-coded three to five, and then to six, makes each machine a separate sweep through the motion core, and the variants cannot be combined because each owns its own copy of that core.

Generalising instead puts the machine-specific part in a declaration and leaves the shared part upstream-shaped, working for three axes as well as for six. One repository, one build and one firmware serve every machine, and `printer.cfg` selects between them. Stock Klipper kinematics keep running from the same build, which is what makes the point above about upstream's tests true.

Scope

In scope. The axis roster and the interfaces built on it, the kinematics engine and the machine variants it serves, the runtime path a move takes, the configuration surface of each variant, and the conventions that keep the fork rebaseable.

Out of scope. The MCU firmware, the serial protocol and step compression, none of which this architecture changes. Upstream Klipper's own kinematics, which continue to work unmodified. Machine mechanics and electrical design.

1 The problem and the idea

Upstream Klipper plans three kinematic axes – x, y and z – followed by the extruder and any “extra axes”. The number three is not a parameter. It is written into the motion core, the g-code layer, the homing code and roughly twenty other modules as slices (`pos[:3]`), literals (`range(3)`), axis strings (`"xyz"`) and fixed indexes (the extruder at 3).

The Turn machines need five: a cartesian gantry over a bed carried by three independent vertical motors. The previous fork changed every three into a five. That works, but each machine repeats the sweep and the fork diverges further from upstream, which continues to add code assuming three.

This architecture removes the number instead.

> **The idea.** The kinematics class owns the axis roster. It publishes the axis letters, and the toolhead, the g-code layer, homing, probing and bed levelling all read them. A kinematics that declares nothing gets upstream's `"xyz"` , so every stock printer runs at three axes with upstream behaviour, from the same build.

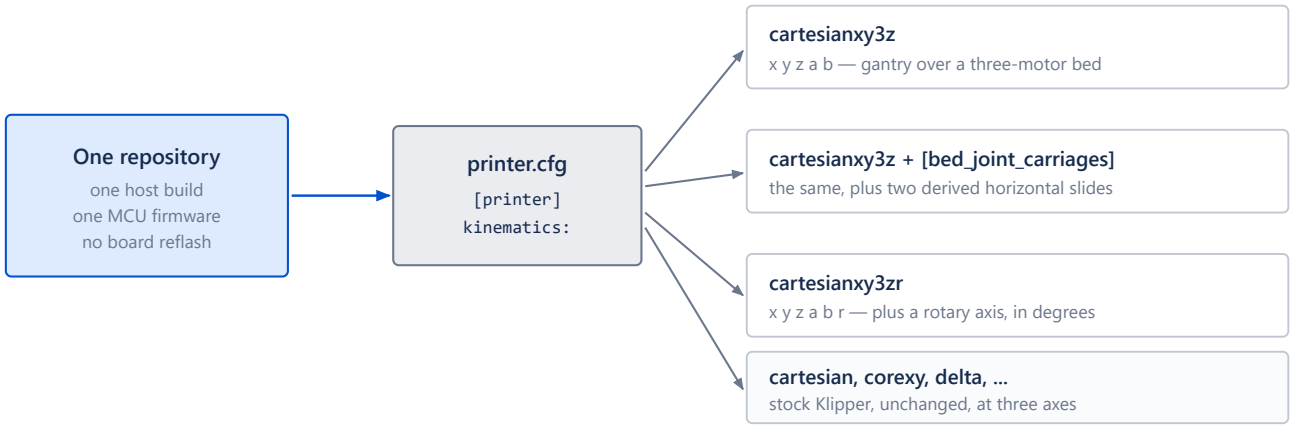


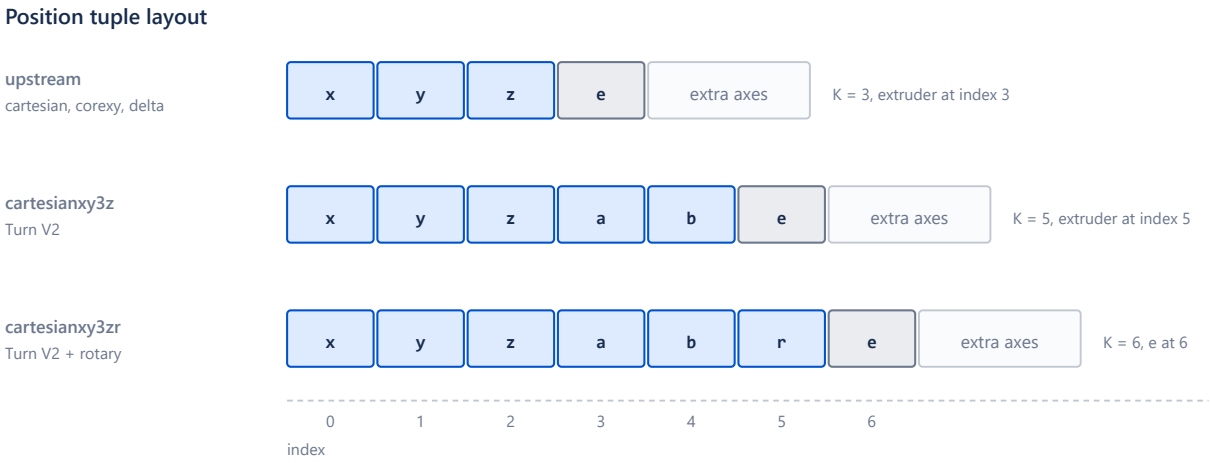
Figure 1. One build, four kinds of machine. The kinematics line in `printer.cfg` is the only thing that differs between them.

Consequences

- **A rebase that converges.** When upstream adds a module assuming three axes, the fix is mechanical and local: ask the toolhead instead of writing a literal.
- **Stock kinematics stay on the same path.** They run through this motion core at three axes rather than around it, so none of them becomes a special case to be worked around.
- **Machines as configuration.** A variant is a roster declaration, not a copy of the kinematics.

2 The axis roster

A kinematics publishes `axis_names`: one lower-case letter per kinematic axis, in order. The toolhead reads it once the kinematics has loaded, sizes the position tuple from it, and places the extruder immediately after the kinematic axes. Everything downstream – the g-code axis map, `M114`, `G28`, homing, status reporting – is derived from the same string.




```
toolhead.get_axis_names() -> "xyzab"
toolhead.get_axis_count() -> 5 toolhead.get_extruder_index() -> 5
```

Figure 2. The extruder index is the axis count. Nothing in the host code may assume a number.

☐ kinematic axes ☐ extruder ☐ extra axes (manual_stepper GCODE_AXIS)

Where the roster is published and consumed

Producer or consumer	Role
`kin.axis_names`	The roster itself, declared by the flavour module.
`kin.distance_groups`	Optional. How a move's length is measured (section 5).
`toolhead._setup_axes()`	Reads both once the kinematics loads; sizes the position tuple; rejects a roster wider than the compiled `KIN_AXES`.
`gcode.Coord`	Re-shaped in place to carry one attribute per axis letter plus `e`, so `position.a` works in macros.
`gcode_move`	Builds the G1 axis map and every axis-naming command from it.
`axis_roster.py`	Answers the machine-shaped questions: bed axes, bed steppers, derived steppers (section 10).

> **Ordering trap.** Config sections are loaded before the toolhead, so a module can capture a reference to `gcode.Coord` before the roster exists. That is why `Coord` is one class object that is re-shaped in place rather than a new class that replaces it. Rebinding the name leaves macros failing with *"Coord object has no attribute 'a'"*.

3 System architecture

The fork touches the host process only. The MCU firmware, the serial protocol and step compression are unchanged, so one firmware serves every machine and no board is reflashed when the variant changes.



Figure 3. The host stack, shaded by ownership as the key below shows. The modification is always the same in kind: replace a fixed axis count with a question to the roster.

fork-only upstream, modified upstream, unchanged

The C boundary

In the compiled helper a coordinate is an array of up to `KIN_AXES` (8) doubles, with `x`, `y` and `z` retained as names so that the stock solvers compile unchanged. Every entry point that carries coordinates takes a count and an array.

```
struct coord { union { struct { double x, y, z; }; double axis[KIN_AXES]; }; };

void   trapq_append(tq, print_time, accel_t, cruise_t, decel_t,
                    int naxes, const double *start_pos, const double *axes_r,
                    start_v, cruise_v, accel);
void   trapq_set_position(tq, print_time, int naxes, const double *pos);
void   itersolve_set_position(sk, int naxes, const double *coord);
double itersolve_calc_position_from_coord(sk, int naxes, const double *coord);
int    trapq_get_kin_axes(void);   /* compiled maximum, checked at startup */
```

`kin_axis.c` is one pass-through solver parameterised by axis index, replacing five copy-pasted ones; `kin_bedjoint.c` solves the derived carriages. Every other solver is upstream's.

4 The life of a move

One `G1` line, followed from the parser to the step queue, on a `cartesianxy3z` machine.

`G1 X100 A5 B5 F3000`

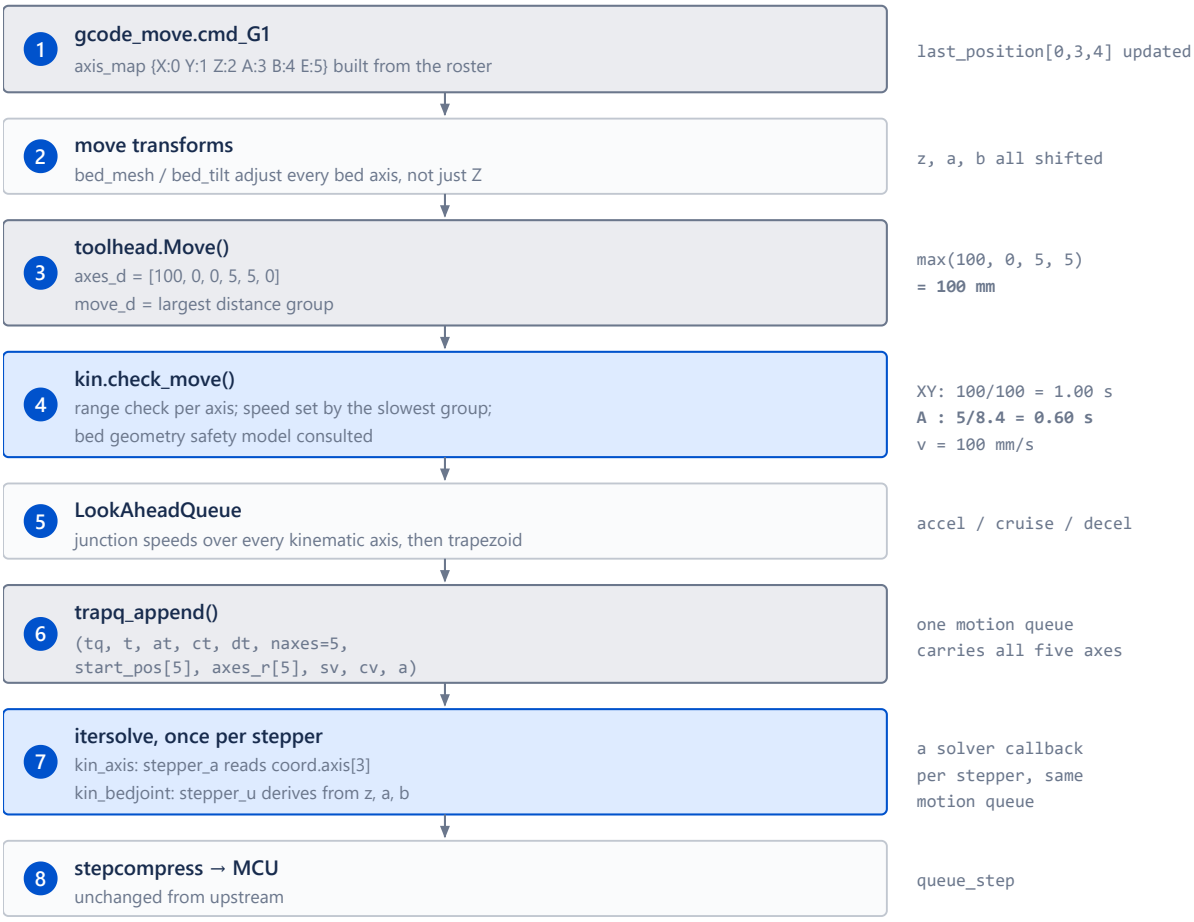


Figure 4. Steps 1, 3, 4 and 7 are where the axis count used to be a literal. Each is now driven by the roster, so the same path serves three, five or six axes.

5 Move distance and speed

Upstream measures a move by the euclidean distance over x, y and z, and a single feed rate then applies to that distance. With three motors under one bed this is the wrong measure: a pure tilt, where Z rises and A falls, has a large euclidean length though each motor travels little, and combining them into one distance makes every tilt unnecessarily slow.

A kinematics may therefore publish `distance_groups`: a list of axis-index tuples. Each group contributes the euclidean norm of its own axes, and the move distance is the **largest** group. Axes in different groups each get the whole move time instead of sharing it.



Figure 5. Each group is timed against its own limit and the slowest one sets the duration of the whole move. Had the bed moved 90 mm instead of 8, the A group would have won and the gantry would have been slowed to match.

The rule in code

```
move_d = max over groups of  ||axes_d[group]||        # toolhead.calc_move_distance
v      = min over groups of  max_velocity[group] * move_d / group_d
a      = min over groups of  max_accel[group]    * move_d / group_d
```

One group covering every axis reproduces upstream exactly, and is the default for a kinematics that publishes nothing. A stock printer's behaviour is therefore unchanged.

> **Axes in no group.** An axis left out of every group is carried along and never lengthens a move. When it is the only axis moving, the largest single axis distance is the fallback, so it still moves and is still limited. This is exactly what `motion_mode: slaved` does to a rotary axis (section 9).

6 The kinematics engine

One engine serves every variant. A flavour module declares a roster and nothing else; the engine builds the rails, homes them, range-checks moves and applies the per-group limits.

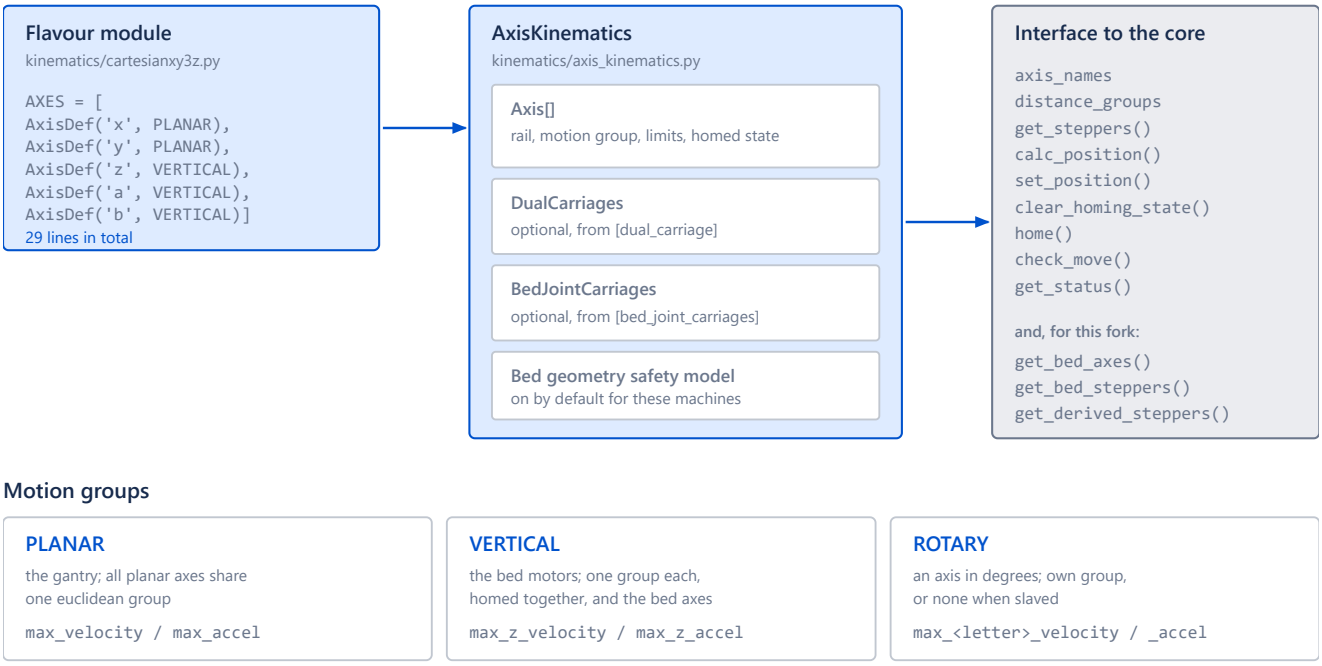


Figure 6. The flavour supplies a list; the engine supplies the behaviour. Adding a machine does not require subclassing.

An `AxisDef` is a letter, a motion group and optionally a config section name, defaulting to `stepper_<letter>`. An axis configured without an `endstop_pin` gets a `BareAxisRail`: usable over its configured range from startup and skipped by `G28`.

7 Homing

A bed on three motors cannot be homed one motor at a time from an unknown pose without tilting it hard against its joints. `G28` therefore runs a coordinated approach first, and only then homes each motor precisely.

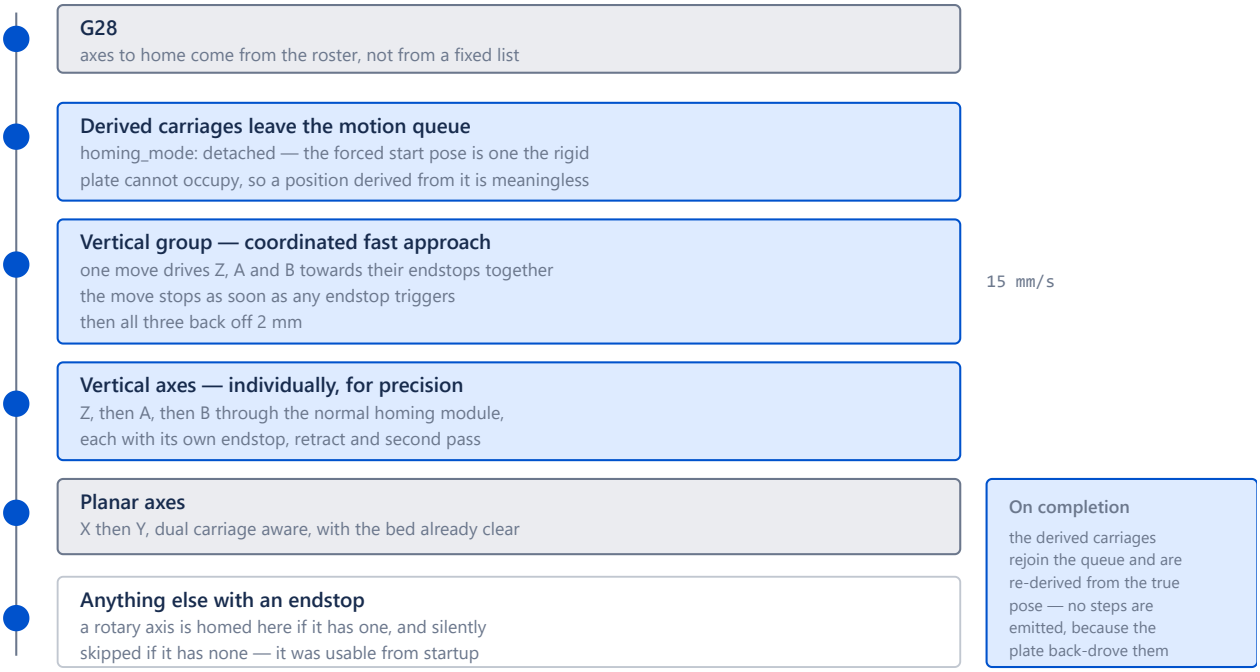
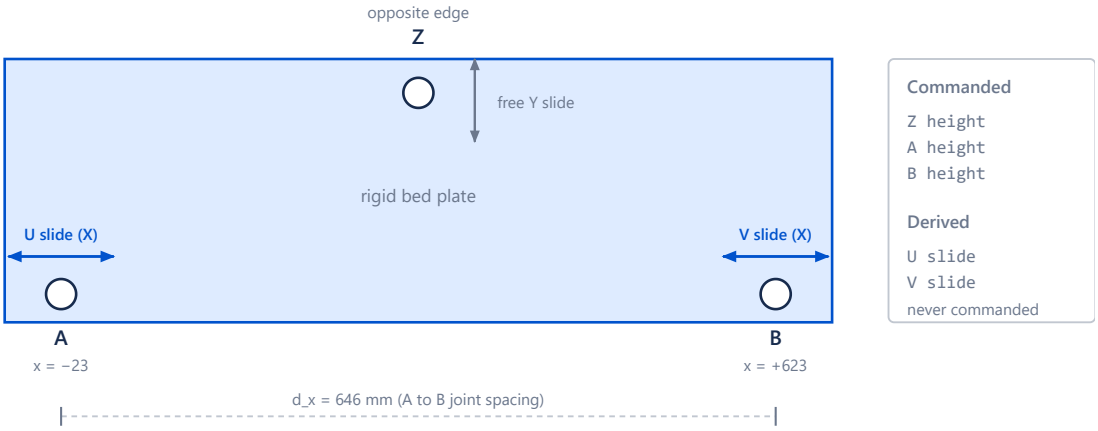


Figure 7. The `G28` sequence. The coordinated phase is a single `HomingMove` across all three bed endstops; if it fails the individual phase still recovers.

8 Derived bed-joint carriages

The bed is a rigid plate on three ball joints. As it tilts, the A and B joints must slide horizontally, and the Z joint slides in the other direction. Plate rigidity fixes those slides completely: they are **not** degrees of freedom. They are modelled the way delta towers are – steppers with a non-trivial position function – rather than as g-code axes.

Plan view of the bed plate



Zero slide is the level-bed home position. Both carriages read 0 with the bed level.

Figure 8. Three commanded heights determine two horizontal slides. Geometry shown for the reference machine.

The solve

Writing the A–B rigid link constraint and eliminating the Z joint's free slide gives a closed form, solved in `klippy/chelper/kin_bedjoint.c`.

```
z_mz = Z - (A + B) / 2                pitch variable
z_ab = B - A                          roll variable
csx  = sqrt(d_x^2 - z_ab^2)           projected A-B separation
U    = mid_x + z_mz*z_ab/csx - csx/2  A joint world X
V    = mid_x + z_mz*z_ab/csx + csx/2  B joint world X
```

The position is recomputed from the current bed axes, so the carriages follow every bed transform – z_tilt, `SET_GCODE_OFFSET`, bed mesh, probe lifts – with no code of their own, and homing them is unnecessary. The g-code is unchanged: there is no U or V word, and the same file runs with or without the carriages fitted.

Why the travel check is sampled

U is nonlinear in the bed pose. A move whose two endpoints are both within the rail can still take a carriage outside it partway through, and the peak rate at the end of a long roll sweep is about twice the average across the move.

Carriage U during one roll sweep: A 20→180, B 180→20



Figure 9. `travel_check: endpoints` sees only the two hollow markers and passes this move. `travel_check: sampled` walks the path and rejects it. The sample count scales with how far the bed pose travels, so an ordinary small adjustment still costs two evaluations.

> **Why the limits are mandatory.** The carriages have no endstop and no encoder. `position_min` and `position_max` on each stepper section are the only thing standing between a commanded bed pose and a ball joint being driven off the end of its rail by the full-power bed actuators. They are required on purpose.

9 The rotary axis

`cartesianxy3zr` adds an axis commanded in degrees and planned in the same trapezoid as the linear axes. Degrees come from the config, not from code: `rotation_distance: 360` with a `gear_ratio` makes one G-code unit one degree.

It is a first-class kinematic axis rather than an upstream "extra axis", because extra axes are excluded from the move distance, cannot be homed by `G28`, and force a full stop around a move that only rotates.

Two configurable choices

Option	Values	Effect
`endstop_pin` `[stepper_r]`	present absent	**Present:** `G28` homes R like any other axis, with `position_endstop`, `homing_speed` and the rest. **Absent:** R has no home reference. It is usable over its configured range from startup, `G28` skips it, and `SET_KINEMATIC_POSITION R=<angle>` redefines its zero.
`motion_mode` `[stepper_r]`	coordinated slaved	**coordinated** (default): R has its own share of the move time, so a combined move is as slow as its slowest part. **slaved:** the linear axes carry R along and it never lengthens a combined move. R moving on its own still works, and is still held to its own limits in both modes.

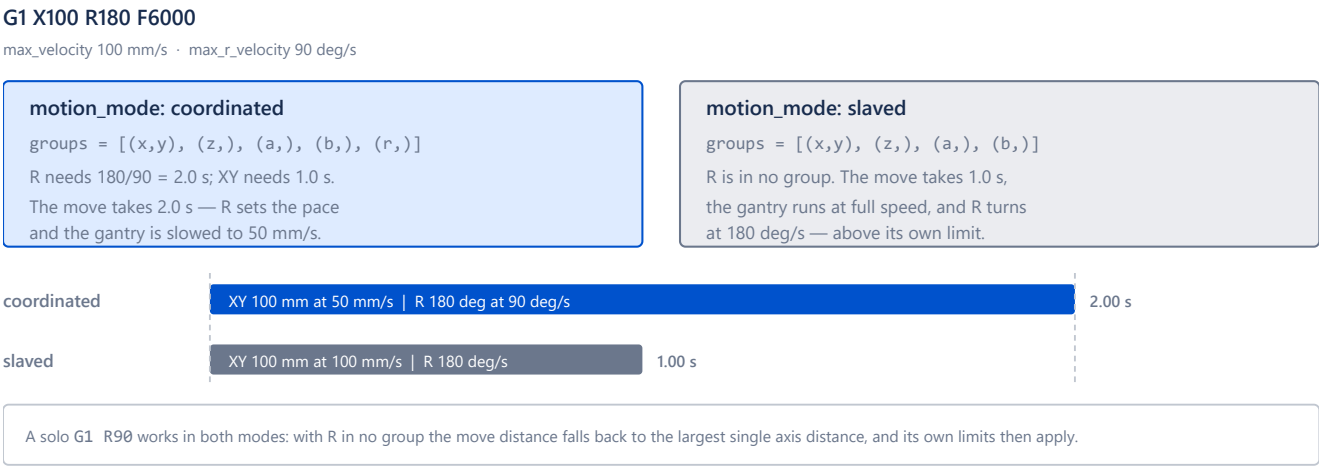
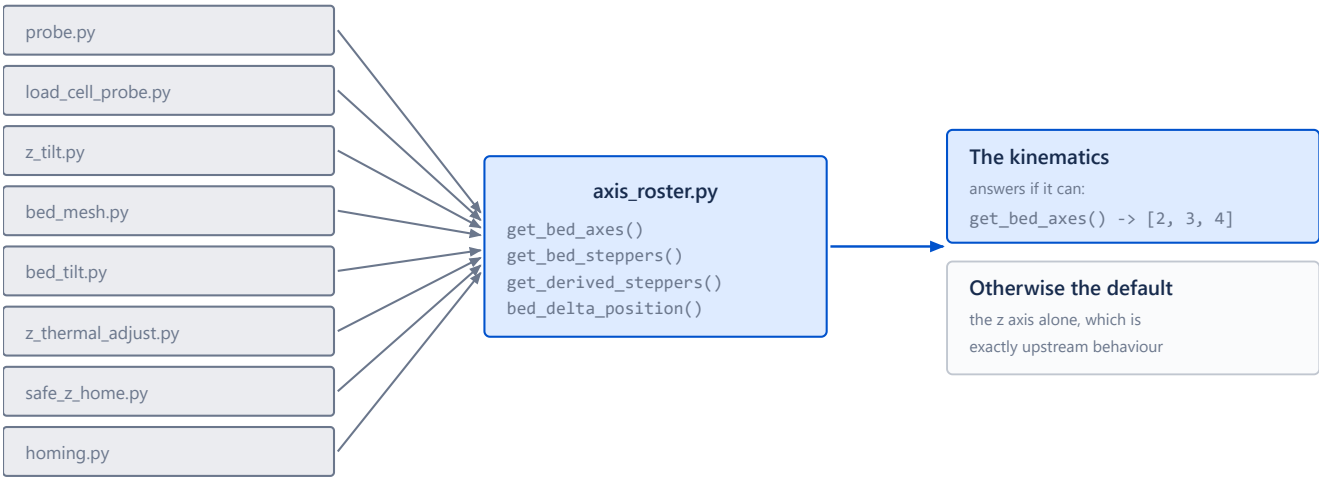


Figure 10. Slaved motion keeps the linear path at full speed and lets the rotary rate follow it; coordinated motion keeps the rotary inside its limit and slows the path to match.

10 Bed queries

Probing and bed levelling assume throughout that the bed is the Z axis. In the previous fork that assumption was rewritten as “axes 2, 3 and 4” in seven separate modules, each copy a place for the next machine to break.



Derived steppers are excluded from the bed stepper set, so z_tilt and probing never mistake a bed-joint carriage for a bed motor.

Figure 11. Seven modules, one question, one answer. A kinematics that says nothing gets the Z axis, so a stock printer keeps upstream behaviour without a special case.

The same helper provides the two position utilities these modules need. `bed_delta_position()` shifts every bed axis by a delta. `bed_target_position()` moves the bed to an absolute height while carrying the other motors by the same amount, which preserves a `Z_TILT_ADJUST` correction across a probe lift.

11 Configuration

Choosing a flavour

Machine	kinematics	Extra sections
Gantry over a three-motor bed	`cartesianxy3z`	—
… with the bed-joint carriages	`cartesianxy3z`	`[bed_joint_carriages]`, `[stepper_u]`, `[stepper_v]`
… with a rotary axis	`cartesianxy3zr`	`[stepper_r]`
… with both	`cartesianxy3zr`	all of the above
Any stock printer	`cartesian`, `corexy`, …	— nothing in this document applies

Baseline

```
[printer]
kinematics: cartesianxy3z
max_velocity: 100
max_accel: 3000
max_z_velocity: 8.4        # shared by Z, A and B
max_z_accel: 300

[stepper_x]                # and stepper_y, stepper_z, stepper_a, stepper_b
...
```

`max_z_velocity` and `max_z_accel` live on `[printer]` in this fork, not on a kinematics section, and apply to every vertical axis. `[dual_carriage]` works as upstream, on X or Y.

Rotary axis

```
[printer]
kinematics: cartesianxy3zr
max_r_velocity: 90         # degrees/s   - the option name follows the letter
max_r_accel: 800           # degrees/s^2

[stepper_r]
step_pin: PXX
dir_pin: !PXX
enable_pin: !PXX
microsteps: 16
rotation_distance: 360     # with gear_ratio, one G-code unit = one degree
gear_ratio: 6:1
position_min: -100000
position_max: 100000
motion_mode: coordinated   # or slaved
#endstop_pin: ^PXX         # add to make G28 home it
```

Derived bed-joint carriages

```
[bed_joint_carriages]
a_joint_home_x: -23        # world X of each ball joint with the bed level
b_joint_home_x: 623
travel_check: sampled      # or endpoints
homing_mode: detached      # or attached
#max_velocity: 8.4         # defaults to max_z_velocity
#max_accel: 300            # defaults to max_z_accel
#carriages: stepper_u, stepper_v

[stepper_u]                # and stepper_v
step_pin: PXX
dir_pin: !PXX
enable_pin: !PXX
microsteps: 16
rotation_distance: 5
position_min: -120         # REQUIRED - the only guard on the rail
position_max: 120
```

Option	Values	Effect
`travel_check`	**sampled** (default) endpoints	Sampled walks the carriage path along the move, catching an interior rail overrun and the true peak rate. Endpoints evaluates only the two ends, which is cheaper and matches the first implementation.
`homing_mode`	**detached** (default) attached	Detached takes the carriages off the motion queue while the bed homes; the plate back-drives them and they are re-derived afterwards without emitting steps. Attached leaves them on the queue, following the commanded pose through the homing moves.

Bed geometry safety checks

Both flavours run a bed-joint clearance model that rejects a move which would collide the extruder with the bed frame. It is **on by default**; no config section is required. Add `[CartesianXY3ZSafety]` to override the geometry or to switch the checks off with `disable_safety_checks: True`. It provides `CHECK_CARTESIANXY3Z_SAFETY` and `PREVIEW_MOVE`. The checks are suspended during homing and probing, because the end position of such a move is a travel limit rather than a point the bed will reach.

WORKED EXAMPLES IN THE REPOSITORY

- `config/example-cartesianxy3z-full.cfg` – the baseline machine
- `config/example-Turn-V2.cfg` – the production machine
- `config/example-cartesianxy3zr.cfg` – the rotary axis
- `config/example-bed-joint-carriages.cfg` – the derived carriages
- `sim/printer.sim.rotary.cfg`, `sim/printer.sim.uvderived.cfg` – simulation profiles, written as thin overlays that include the baseline

12 What changes for the operator

Area	Difference from upstream
`G1`, `G92`, `G2` / `G3`, `SET_GCODE_OFFSET`	Accept a word per kinematic axis: `G1 X10 Y10 Z5 A5 B5`, and `R45` on the rotary flavour. Arc moves interpolate every kinematic axis outside the arc plane.
`M114`, `GET_POSITION`	Report every axis in the roster.
`SET_VELOCITY_LIMIT`	Adds `Z_VELOCITY=` and `Z_ACCEL=` for the shared vertical limits.
`SET_KINEMATIC_POSITION`	Takes a parameter per axis; `SET_HOMED` and `CLEAR_HOMED` follow the roster.
`G28`	Bed axes first, coordinated then individually; then X and Y; then anything else with an endstop.
`Z_TILT_ADJUST`	Corrects the three bed motors in one coordinated move, then applies session g-code offsets so they all report the same coordinate. The offsets reset on the next `G28`.
Probing	Lifts and descents move every bed motor by the same delta, so a tilt correction survives them. `probe_travel_limit` (default `max(-1.5, position_min)`) bounds a descent.
Extrusion	`max_extrude_cross_section` is **not enforced**, only logged: the move distance includes bed travel, so the ratio is not a meaningful measure on this machine.
Status	`toolhead.position`, `axis_minimum` and `axis_maximum` carry every axis; `homed_axes` uses the roster letters. With the carriages fitted, `bed_joint_carriages` reports their positions.

Known limitation: manual probing moves the Z motor alone. `manual_probe.py` jogs the bed with `manual_move([None, None, z])`, which drives the z axis and leaves the other bed motors behind, so `PROBE_CALIBRATE`, `Z_ENDSTOP_CALIBRATE` and every `TESTZ` jog tilt the plate slightly and calibrate the offset at a tilted pose. The module is untouched by the fork, so it does not go through the roster; building its coordinate with `axis_roster.bed_target_position()` inside `move_z()` would bring it in line. The same pattern is in `probe_eddy_current.py`, `temperature_probe.py` and the bed mesh scan helper, which matter only if those modules are enabled.

Macros. `printer.toolhead.position.a` and `axis_maximum.b` work because the `Coord` class carries one attribute per roster letter. On a stock three axis printer those attributes do not exist, which is the correct answer rather than a silent zero.

13 Extending and maintaining

Adding a machine flavour

1. Add a module in `klippy/kinematics/` with an `AXES` list and a `load_kinematics` that calls `AxisKinematics`. Do not subclass an existing flavour.
2. Add a simulation profile in `sim/` as a thin overlay that includes `printer.sim.cfg` and overrides only what differs.
3. Add a worked example in `config/`.

Adding a derived stepper

1. Write a C solver in `klippy/chelper/` that reads the axes it depends on from `move_get_coord()`, and register it in `chelper/__init__.py`.
2. Add a feature module shaped like `bed_joint_carriages.py`: allocate the steppers, re-derive them in `set_position`, range-check and rate-limit them in `check_move`, and report them through `get_derived_steppers()` so that probing and levelling ignore them.

Surviving the next upstream rebase

`FORK_CHANGES.md` carries the full checklist. The substance of it is four greps and one rule.

Upstream pattern	Fork equivalent
`pos[:3]`, `pos[3:]`, `range(3)`	`[:naxes]`, `[naxes:]`, `range(naxes)`
`"xyz"`, `'XYZ'`, `'XYZE'`	`toolhead.get_axis_names()` (plus `'E'`)
extruder index `3`	`toolhead.get_extruder_index()`
“the Z axis carries the bed”	`axis_roster.get_bed_axes(printer)`
“these are the Z steppers”	`axis_roster.get_bed_steppers(printer)`
coordinate-carrying C calls	a count and an array

Three literals inside modified files are deliberate and carry a comment saying so: the bed mesh move splitter measures the x/y/z *path* it slices, the bed mesh transform reads x and y because the mesh is a height over that *surface*, and `gcode_arcs` requires x, y and z because an arc *plane* is defined over them. Do not widen them to the roster.

> **The rule.** Never write a literal kinematic axis index. If a new upstream module indexes a position, decide whether it means the kinematic axes – then convert it – or something else, and leave it alone. A new upstream kinematics needs no work: it runs at three axes.

Where things live

Path	Role
`klippy/axis_roster.py`	fork-only — the roster and bed queries
`klippy/kinematics/axis_kinematics.py`	fork-only — the engine
`klippy/kinematics/cartesianxy3z.py`	fork-only — flavour
`klippy/kinematics/cartesianxy3zr.py`	fork-only — flavour
`klippy/kinematics/bed_joint_carriages.py`	fork-only — derived carriages
`klippy/kinematics/cartesianxy3z_safety.py`	fork-only — bed clearance model
`klippy/chelper/kin_axis.c`	fork-only — pass-through solver
`klippy/chelper/kin_bedjoint.c`	fork-only — carriage solver
`klippy/toolhead.py`, `gcode.py`, `gcode_move.py`, `stepper.py`, `extras/homing.py`	upstream, modified for the roster
`docs/Turn_Kinematics.md`	this document, as text
`FORK_CHANGES.md`	the file-by-file maintenance ledger